

Tutorial - Semana 3 - Encontro 2

Introdução

No Encontro 1 você resolveu a aparência (material) e a geometria (Landscape) do nível de teste. Mas dois problemas de renderização moderna ainda não foram tratados: como exibir geometria detalhada sem comprometer performance, e como simular a luz que rebate entre superfícies sem depender de um cálculo prévio (bake). Este encontro fecha o Módulo 1 respondendo a essas duas perguntas com Nanite e Lumen, e em seguida transforma tudo o que foi construído até aqui — `BP_Player`, Enhanced Input, material, Landscape — no primeiro executável real do Vertical Slice, através do Packaging.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar Nanite (geometria virtualizada) e Lumen (iluminação global dinâmica) como soluções da Unreal para dois problemas universais de renderização moderna.
- Explicar o pipeline de Packaging como transformação de um projeto editável em um build executável.
- Ativar e ajustar Nanite/Lumen no nível de teste e gerar o primeiro build empacotado do Vertical Slice.

Resultado Esperado ao Final da Semana

Um build executável do Vertical Slice, gerado fora do Editor, com Nanite e Lumen ativos no `Map_Exploration` (material e Landscape do Encontro 1) e o `BP_Player` controlável dentro desse executável pelo esquema de Enhanced Input configurado na Semana 2. Este é o Checkpoint de encerramento do Módulo 1 — Aprender a Ferramenta.

Pré-requisitos

- Ter concluído o Encontro 1: material `M_Ground` e terreno via Landscape prontos no `Map_Exploration`.

- Projeto com `BP_Player` funcional desde a Semana 2, controlável exclusivamente por Enhanced Input.

Antes de começar

O que você deverá possuir antes desta semana

- O projeto do Encontro 1, aberto no Unreal Editor 5.6, com o material e o Landscape prontos e o `BP_Player` controlável sobre o terreno.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Os mesmos assets já presentes no projeto (material `M_Ground`, texturas do Kenney). Nenhum asset novo é adicionado neste encontro.

Projeto esperado

O mesmo projeto do Encontro 1, pronto para receber a ativação de Nanite/Lumen e a configuração de Packaging.

Parte 1 — Nanite e Lumen: dois problemas universais de renderização

Objetivo

Compreender Nanite e Lumen como soluções para dois problemas distintos — complexidade geométrica e iluminação indireta —, e ativá-los/ajustá-los no nível de teste.

Conceito

Toda engine moderna enfrenta o mesmo par de problemas ao renderizar um mundo tridimensional. O primeiro é a **complexidade geométrica**: quanto mais detalhado um modelo (mais polígonos), mais caro é renderizá-lo a cada quadro. A solução tradicional é o uso de **LODs** (Levels of Detail) criados manualmente — versões simplificadas do mesmo modelo, trocadas conforme a distância da câmera. O **Nanite** resolve esse problema de forma diferente: ele virtualiza a geometria, exibindo apenas o nível de detalhe necessário a cada pixel da tela, sem que o desenvolvedor precise criar LODs manuais para a maioria dos assets.

O segundo problema é a **iluminação indireta**: como simular a luz que rebate entre superfícies (por exemplo, a luz que entra por uma janela e ilumina indiretamente uma parede oposta), sem que isso custe caro demais para calcular em tempo real. A solução tradicional é o **bake de lightmaps** — um cálculo prévio, feito antes de rodar o jogo, que fica desatualizado se a geometria ou a luz mudarem depois. O **Lumen** resolve esse problema com iluminação global totalmente dinâmica, calculada em tempo real, reagindo a mudanças de geometria e luz sem exigir bake algum.

Nanite resolve geometria; Lumen resolve luz. São dois problemas independentes, e cada um tem sua própria configuração dentro do projeto — não devem ser confundidos como uma única "opção de gráficos melhores".

Passo a passo

1. Abrir Edit > Project Settings e localizar a seção "Rendering".
2. Na subseção "Global Illumination", confirmar que o método está definido como "Lumen" (ativado por padrão em projetos novos da Unreal 5.6; caso contrário, selecionar "Lumen" no campo "Dynamic Global Illumination Method").
3. Na subseção "Reflections", confirmar que o "Reflection Method" também está definido como "Lumen", para que reflexos também usem o mesmo sistema de iluminação dinâmica.
4. Fechar Project Settings e abrir o `Map_Exploration`.
5. Selecionar o Landscape criado no Encontro 1 e, no painel Details, localizar a propriedade relacionada a Nanite (em versões recentes, Landscape já se beneficia de otimizações internas equivalentes; para Static Meshes adicionais do nível, localizar a categoria "Nanite Settings" no Static Mesh Editor e habilitar "Enable Nanite Support").
6. Testar em modo Play e observar a iluminação do nível: sombras e reflexos indiretos devem responder dinamicamente caso uma luz do nível seja movida ou sua intensidade alterada durante o teste.
7. Ajustar a intensidade de uma Directional Light (ou Sky Light) presente no nível e observar, em tempo real, a mudança na iluminação indireta das superfícies próximas — sem necessidade de qualquer bake.

Resultado esperado

O `Map_Exploration` com Lumen ativo, respondendo em tempo real a mudanças de iluminação, e com Nanite disponível para os assets do nível que o suportam, sem necessidade de LODs manuais para esses casos.

Verificando

Mova ou altere a intensidade de uma luz do nível em modo Play (ou em modo Editor, observando o viewport em tempo real) e confirme que a iluminação indireta das superfícies próximas muda imediatamente, sem exigir recompilação de lightmaps.

Problemas comuns

- **Confundir Nanite com "mais polígonos permitidos":** Nanite não aumenta o limite de polígonos por si só; ele muda como a geometria é processada e exibida, virtualizando o detalhe necessário por pixel.
- **Achar que Lumen substitui a necessidade de posicionar luzes corretamente:** Lumen calcula a propagação da luz existente na cena; ele não gera luz onde não há nenhuma fonte luminosa.
- **Ativar Nanite em um asset muito simples:** para geometria já muito leve (como formas de graybox), o ganho de Nanite é imperceptível; o conceito importa mais que o ajuste fino neste estágio do curso.

Boas práticas

Ajuste a iluminação do nível observando o resultado em tempo real (Lumen permite isso), em vez de ajustar valores "no escuro" e só verificar depois de um bake — esse fluxo de iteração rápida é uma das vantagens centrais da iluminação dinâmica sobre o bake tradicional.

Comparação com Unity

Nanite não possui equivalente direto na Unity: o caminho tradicional (e ainda majoritário na Unity) é a criação manual de LODs e o controle cuidadoso de contagem de polígonos por cena, embora a Unity venha explorando soluções próprias de geometria em alta densidade em pipelines mais recentes. Lumen corresponde, em intenção, ao GI dinâmico da Unity (URP/HDRP) ou a soluções de terceiros, mas a Unreal entrega Lumen ativado por padrão em novos projetos, enquanto a Unity historicamente depende mais do bake de lightmaps (Progressive Lightmapper) para iluminação indireta de alta qualidade.

Parte 2 — Packaging: do projeto ao executável

Objetivo

Compreender o Packaging como a etapa que transforma um projeto editável em um produto executável, e gerar o primeiro build do Vertical Slice.

Conceito

Até agora, tudo o que foi construído — `BP_Player`, Enhanced Input, material, Landscape, Nanite, Lumen — só existe dentro do Unreal Editor. Nenhuma engine entrega um jogo jogável fora do editor sem uma etapa de conversão explícita: o **Packaging**. Essa etapa compila o projeto para uma plataforma-alvo específica (neste curso, Windows), empacota todos os assets necessários, e gera um executável autocontido, capaz de rodar em qualquer máquina compatível sem precisar do Editor instalado.

O Packaging é o momento em que o trabalho de todo o Módulo 1 deixa de ser um protótipo interno e passa a existir como um artefato concreto e distribuível — ainda que, neste estágio, seja apenas um personagem controlável em um graybox com material e terreno básicos.

Passo a passo

1. Confirmar que o `Map_Exploration` está definido como o mapa padrão do projeto: Edit > Project Settings > Maps & Modes > "Editor Startup Map" e "Game Default Map", ambos apontando para `Map_Exploration`.
2. Ir até Platforms > Windows (ou o menu equivalente "Package Project" na versão 5.6) na barra de ferramentas principal do Editor.
3. Selecionar a plataforma-alvo "Windows".
4. Escolher a configuração de build "Development" (adequada para testes durante o curso, mais rápida de gerar que "Shipping").
5. Selecionar uma pasta de destino vazia no computador para receber os arquivos do build (por exemplo, uma pasta `Buils/Semana03` fora da pasta do projeto).
6. Iniciar o processo de Packaging e aguardar a conclusão — o tempo de build varia conforme o tamanho do projeto e o hardware da máquina, podendo levar alguns minutos.
7. Ao concluir, abrir a pasta de destino e localizar o executável gerado (arquivo `.exe` com o nome do projeto).
8. Executar o `.exe` diretamente, fora do Unreal Editor, e testar a movimentação do `BP_Player` com o esquema de Enhanced Input configurado na Semana 2.

Resultado esperado

Um executável autocontido, localizado na pasta de destino escolhida, que roda fora do Unreal Editor e no qual o `BP_Player` se move, olha ao redor e pula normalmente sobre o terreno com material criado no Encontro 1, com Nanite e Lumen ativos.

Verificando

Feche completamente o Unreal Editor antes de testar o executável, garantindo que o build realmente roda de forma independente, sem depender de nenhum processo do Editor em segundo plano.

Problemas comuns

- **Mapa padrão não configurado corretamente:** se "Game Default Map" não apontar para `Map_Exploration`, o build pode abrir em um nível vazio ou incorreto; revise Project Settings antes de iniciar o Packaging.
- **Build demora mais que o esperado:** normal para a primeira compilação do projeto (compilação de shaders inclusa); iniciar o Packaging o quanto antes no laboratório evita perda de tempo no Checkpoint.
- **Referências quebradas durante o build:** geralmente causadas por assets movidos ou renomeados sem atualizar referências; verificar o log de Packaging (acessível pelo próprio Editor) para identificar o asset problemático.
- **Executável não abre ou fecha imediatamente:** confirme que a plataforma-alvo selecionada corresponde ao sistema operacional da máquina de teste.

Boas práticas

Reserve tempo de build no cronograma de qualquer entrega futura do curso — builds não são instantâneos, e deixar o Packaging para o último minuto de qualquer Checkpoint é um risco técnico evitável, não apenas nesta semana.

Comparação com Unity

Packaging corresponde ao Build da Unity (File > Build Settings): ambos resolvem o mesmo problema de gerar um executável a partir do projeto editável, com configuração de plataforma-alvo equivalente em intenção (Windows, Mac, consoles, mobile). A diferença mais perceptível no dia a dia é o tempo de compilação de shaders na primeira build de um projeto Unreal, algo que a Unity também enfrenta de forma distinta conforme o pipeline de renderização escolhido.

Ao final da semana

Ao final da Semana 3 (Encontros 1 e 2), o projeto deve possuir: um nível de teste com material simples e terreno via Landscape (Encontro 1), Nanite e Lumen ativos, e um primeiro build executável do Vertical Slice, gerado via Packaging, com o `BP_Player` (Character + Enhanced Input, construído nas Semanas 1 e 2) funcional dentro desse executável. Isso corresponde à conclusão da linha "Renderização moderna e build" do roadmap do Módulo 1 no `PROJECT_ARCHITECTURE.md`, e ao produto formal do módulo: "primeiro build executável, com o jogador explorando um graybox do ambiente externo". Este é também o encerramento da Unidade I — Aprender a Ferramenta.

Desafio

Não há desafio de liberdade de solução neste encontro. O instrumento avaliativo do encontro é o Checkpoint de encerramento do Módulo 1 (Rubrica 3 — Checkpoints), não um desafio técnico de solução aberta — a prioridade pedagógica é garantir que todo grupo chegue a um build funcional, coerente com a autonomia muito baixa do Módulo 1 (`PEDAGOGICAL_RULES.txt`).

Checklist

- ☐ Lumen ativo como método de Global Illumination e Reflections no projeto
- ☐ Nanite habilitado nos assets do nível que o suportam
- ☐ Iluminação do nível respondendo em tempo real a mudanças, sem necessidade de bake
- ☐ "Editor Startup Map" e "Game Default Map" apontando para `Map_Exploration`
- ☐ Build gerado com sucesso via Packaging (configuração Development, plataforma Windows)
- ☐ Executável testado fora do Editor, com `BP_Player` controlável por Enhanced Input dentro do build

Glossário

- **Nanite:** sistema de geometria virtualizada da Unreal, que exibe o detalhe necessário a cada pixel sem exigir LODs manuais.
- **Lumen:** sistema de iluminação global dinâmica da Unreal, que calcula luz indireta em tempo real sem necessidade de bake prévio.
- **LOD (Level of Detail):** versão simplificada de um modelo, tradicionalmente trocada conforme a distância da câmera.

- **Bake (de iluminação):** cálculo prévio de iluminação indireta, armazenado em lightmaps, que não reage a mudanças em tempo real.
- **Packaging:** processo de compilar e empacotar um projeto editável em um executável autocontido para uma plataforma-alvo específica.
- **Build:** o produto executável gerado pelo processo de Packaging.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Nanite Virtualized Geometry. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/nanite-virtualized-geometry>.
- EPIC GAMES. **Unreal Engine 5 Documentation** — Lumen Global Illumination and Reflections. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/lumen-global-illumination-and-reflections>.
- EPIC GAMES. **Unreal Engine 5 Documentation** — Packaging Your Project. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/packaging-your-project>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Nanite, Lumen e Packaging. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — Build Settings, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Nanite, Lumen e Packaging.